Лабораторная работа №4
«Система видеонаблюдения на базе веб-камеры»

1. Цель работы

Изучить архитектуру подсистем видеозахвата операционных систем Windows и Linux; освоить программные интерфейсы для обнаружения, идентификации и управления веб-камерами; приобрести практические навыки разработки приложений на C++ для получения, отображения и сохранения видеопотока в реальном времени с использованием низкоуровневых системных API (DirectShow / Media Foundation в Windows, Video4Linux2 в Linux) без применения высокоуровневых библиотек-обёрток (OpenCV, FFmpeg API, DirectShow.NET); научиться реализовывать фоновый режим наблюдения, журналирование событий и управление горячими клавишами.

2. Задачи

1. Изучить архитектуру подсистем видеозахвата: DirectShow / Media Foundation (Windows) и Video4Linux2 (Linux).
2. Изучить механизмы обнаружения и перечисления устройств видеозахвата (SetupAPI / CLSID_SystemDeviceEnum в Windows; sysfs / udev / V4L2 API в Linux).
3. Освоить механизмы получения информации об устройстве: наименование, производитель, поддерживаемые разрешения, частоты кадров, форматы пикселей.
4. Реализовать захват видеопотока с веб-камеры и его отображение в окне приложения в реальном времени.
5. Реализовать создание фотоснимков (захват отдельного кадра) с сохранением в файл (BMP / JPEG / PPM).
6. Реализовать запись видеофрагментов с сохранением в файл (AVI / MJPEG / MP4).
7. Обеспечить отображение даты и времени создания материалов, автоматическое сохранение в выбранный каталог.
8. Реализовать ведение журнала сеансов наблюдения (подключение, отключение, захват, ошибки).
9. Реализовать контроль состояния камеры (подключена / отключена) с индикацией в GUI.
10. Реализовать скрытый режим работы (фоновый захват по горячей клавише без отображения главного окна).
11. Разработать графический пользовательский интерфейс приложения в соответствии с установленным в ЛР №1 стилем оформления.

Интерфейсы и устройства вычислительных машин

3. Теоретический материал

3.1. Общие сведения о подсистемах видеозахвата

Видеозахват — процесс получения непрерывного потока кадров (frames) с устройства видеоввода (веб-камеры, ТВ-тюнера, платы захвата) и его дальнейшая обработка или сохранение. Современные ОС предоставляют стандартизированные интерфейсы, абстрагирующие специфику конкретного оборудования:

- **Windows:** DirectShow (устаревающий, но широко поддерживаемый) и Media Foundation (современный API, рекомендуемый Microsoft с Windows Vista/7);
- **Linux:** Video4Linux2 (V4L2) — универсальный драйверный интерфейс для устройств видео и радио.

В рамках данной работы запрещается использование высокоуровневых библиотек (OpenCV, FFmpeg/libav, GStreamer, DirectShow.NET), поскольку целью является изучение непосредственного взаимодействия с системными API.

3.2. Windows: DirectShow

DirectShow — фреймворк Microsoft для захвата, обработки и воспроизведения мультимедиа. Построен на архитектуре фильтров (Filter Graph).

3.2.1. Перечисление устройств захвата

```
#include <dshow.h>

// Создание системного перечислителя
ICreateDevEnum* pDevEnum = nullptr;
CoCreateInstance(CLSID_SystemDeviceEnum, nullptr, CLSCTX_INPROC,
                 IID_ICreateDevEnum, (void**)&pDevEnum);

// Получение перечислителя категории Video Capture
IEnumMoniker* pEnum = nullptr;
pDevEnum->CreateClassEnumerator(CLSID_VideoInputDeviceCategory,
&pEnum, 0);

// Перебор устройств
IMoniker* pMoniker = nullptr;
while (pEnum->Next(1, &pMoniker, nullptr) == S_OK) {
    IPropertyBag* pPropBag = nullptr;
    pMoniker->BindToStorage(nullptr, nullptr, IID_IPropertyBag,
                            (void**)&pPropBag);
    VARIANT varName;
    VariantInit(&varName);
    pPropBag->Read(L"FriendlyName", &varName, nullptr);
```

Интерфейсы устройств вычислительных машин

```
    // varName.bstrVal — имя устройства
    VariantClear(&varName);
    pPropBag->Release();
    pMoniker->Release();
}
```

3.2.2. Получение капабилити (IAMStreamConfig)

После создания графа захвата (Capture Graph) через ICaptureGraphBuilder2 можно получить интерфейс IAMStreamConfig для настройки потока:

```
IAMStreamConfig* pConfig = nullptr;
pBuild->FindInterface(&PIN_CATEGORY_CAPTURE, &MEDIATYPE_Video,
                      pCaptureFilter, IID_IAMStreamConfig,
                      (void**)&pConfig);

int iCount = 0, iSize = 0;
pConfig->GetNumberOfCapabilities(&iCount, &iSize);

for (int i = 0; i < iCount; i++) {
    AM_MEDIA_TYPE* pmt = nullptr;
    VIDEO_STREAM_CONFIG_CAPS caps;
    pConfig->GetStreamCaps(i, &pmt, (BYTE*)&caps);
    // caps.InputSize — разрешение
    // caps.MaxFrameInterval / MinFrameInterval — интервал кадров
(100 нс)
    // pmt->subtype — формат пикселей (MEDIASUBTYPE_RGB24,
MEDIASUBTYPE_MJPG)
    DeleteMediaType(pmt);
}
```

3.2.3. Захват и отображение

Для отображения видеопотока используется фильтр рендеринга (Video Renderer или VMR-7/VMR-9/EVR). Для получения отдельных кадров в память используется Sample Grabber (ISampleGrabber) или пользовательский фильтр (push source).

3.3. Windows: Media Foundation

Media Foundation — современный API мультимедиа, рекомендуемый для новых приложений.

3.3.1. Перечисление источников видео

```
#include <mfapi.h>
#include <mfidl.h>

IMFActivate** ppDevices = nullptr;
UINT32 count = 0;
MFEnumDeviceSources(MF_DEVSOURCE_ATTRIBUTE_SOURCE_TYPE_VIDCAP_GUID,
                    &ppDevices, &count);
```

Интерфейсы и устройства вычислительных машин

```
for (UINT32 i = 0; i < count; i++) {
    WCHAR* name = nullptr;
    UINT32 length = 0;
    ppDevices[i]->GetAllocatedString(
        MF_DEVSOURCE_ATTRIBUTE_FRIENDLY_NAME, &name, &length);
    // name — имя устройства
    CoTaskMemFree(name);
    ppDevices[i]->Release();
}
CoTaskMemFree(ppDevices);
```

3.3.2. Получение типов медиа (разрешение, FPS, формат)

```
IMFMediaSource* pSource = nullptr;
ppDevices[index]->ActivateObject(IID_PPV_ARGS(&pSource));

IMFPresentationDescriptor* pPD = nullptr;
pSource->CreatePresentationDescriptor(&pPD);

IMFStreamDescriptor* pSD = nullptr;
BOOL selected = FALSE;
pPD->GetStreamDescriptorByIndex(0, &selected, &pSD);

IMFMediaTypeHandler* pHandler = nullptr;
pSD->GetMediaTypeHandler(&pHandler);

DWORD typeCount = 0;
pHandler->GetMediaTypeCount(&typeCount);
for (DWORD i = 0; i < typeCount; i++) {
    IMFMediaType* pType = nullptr;
    pHandler->GetMediaTypeByIndex(i, &pType);
    UINT32 width = 0, height = 0;
    MFGetAttributeSize(pType, MF_MT_FRAME_SIZE, &width, &height);
    UINT32 num = 0, den = 0;
    MFGetAttributeRatio(pType, MF_MT_FRAME_RATE, &num, &den);
    // num/den — FPS
    pType->Release();
}
```

3.3.3. Захват кадра через Source Reader

```
IMFSourceReader* pReader = nullptr;
MFCreateSourceReaderFromMediaSource(pSource, nullptr, &pReader);

// Установка формата
IMFMediaType* pType = nullptr;
MFCreateMediaType(&pType);
pType->SetGUID(MF_MT_MAJOR_TYPE, MFMediaType_Video);
pType->SetGUID(MF_MT_SUBTYPE, MFVideoFormat_RGB24);
MFSetAttributeSize(pType, MF_MT_FRAME_SIZE, width, height);
pReader->SetCurrentMediaType(MF_SOURCE_READER_FIRST_VIDEO_STREAM,
                             nullptr, pType);
```

Интерфейсы и устройства вычислительных машин

```
// Чтение кадра
IMFSample* pSample = nullptr;
pReader->ReadSample(MF_SOURCE_READER_FIRST_VIDEO_STREAM, 0,
                    nullptr, nullptr, nullptr, &pSample);

IMFMediaBuffer* pBuffer = nullptr;
pSample->ConvertToContiguousBuffer(&pBuffer);
BYTE* pData = nullptr;
DWORD maxLen = 0, curLen = 0;
pBuffer->Lock(&pData, &maxLen, &curLen);
// pData — указатель на RAW кадр в формате RGB24
pBuffer->Unlock();
```

3.3.4. Сохранение фотоснимка

Для сохранения кадра в BMP достаточно сформировать BITMAPFILEHEADER + BITMAPINFOHEADER и записать RAW-данные. Для JPEG — использовать GDI+ (GdipSaveImageToFile) или Media Foundation encoder (IMFSinkWriter).

3.3.5. Запись видео

Для записи видео в Media Foundation используется IMFSinkWriter:

```
IMFSinkWriter* pSinkWriter = nullptr;
MFCreateSinkWriterFromURL(L"output.mp4", nullptr, nullptr,
&pSinkWriter);
// Настройка входного и выходного типа, добавление видеопотока
// Запись сэмплов через pSinkWriter->WriteSample()
```

3.4. Windows: Обнаружение подключения / отключения камеры

Для мониторинга состояния устройства используется механизм **WM_DEVICECHANGE**:

- окно регистрируется на уведомления через RegisterDeviceNotification с GUID_DEVINTERFACE_USB_DEVICE или DBT_DEVTYP_DEVICEINTERFACE;

- при подключении/отключении камеры система рассылает сообщение WM_DEVICECHANGE с параметрами DBT_DEVICEARRIVAL / DBT_DEVICEREMOVECOMPLETE.

3.5. Windows: Скрытый режим и горячие клавиши

3.5.1. Скрытое окно и системный трей

Для скрытого режима используется:

- **Message-only window** (HWND_MESSAGE) — окно без видимого интерфейса, принимающее системные сообщения;

- **Shell_NotifyIcon** — размещение иконки в системном трее с контекстным

Интернет вещей: устройства вычислительных машин

меню;

- скрытие главного окна: ShowWindow(hWnd, SW_HIDE).

3.5.2. Глобальные горячие клавиши

```
// Регистрация горячей клавиши (например, Ctrl+Shift+F12)
RegisterHotKey(hWnd, 1, MOD_CONTROL | MOD_SHIFT, VK_F12);

// В оконной процедуре:
case WM_HOTKEY:
    if (wParam == 1) {
        // Выполнить захват кадра или начать/остановить запись
    }
    break;
```

Примечание: RegisterHotKey требует, чтобы окно обрабатывало сообщения. Для полностью скрытого приложения без GUI можно использовать низкоуровневый хук клавиатуры (SetWindowsHookEx с WH_KEYBOARD_LL), но это требует отдельного потока обработки сообщений.

3.6. Linux: Video4Linux2 (V4L2)

Video4Linux2 (V4L2) — драйверный интерфейс ядра Linux для устройств видеозахвата, радио и VBI. Взаимодействие осуществляется через системные вызовы open, ioctl, mmap, read, close с файлом устройства /dev/videoN.

3.6.1. Обнаружение устройств

В Linux устройства V4L2 представлены файлами /dev/video0, /dev/video1 и т.д. Для перечисления устройств можно:

- сканировать /sys/class/video4linux/ (содержит симлинки на устройства);
- использовать **libudev** (низкоуровневый API): udev_new, udev_enumerate_add_match_subsystem(enumerate, "video4linux");
- открывать /dev/videoN и выполнять VIDIOC_QUERYCAP.

```
#include <linux/videodev2.h>
#include <fcntl.h>
#include <sys/ioctl.h>

int fd = open("/dev/video0", O_RDWR);
struct v4l2_capability cap;
if (ioctl(fd, VIDIOC_QUERYCAP, &cap) == 0) {
    // cap.card – имя устройства (например, "USB Camera")
    // cap.driver – имя драйвера (например, "uvcvideo")
    // cap.bus_info – шина (например, "usb-0000:00:14.0-5")
    // cap.capabilities & V4L2_CAP_VIDEO_CAPTURE – поддержка захвата
}
```

3.6.2. Получение капабилити

Форматы пикселей (FourCC):

Интерфейсы и устройства вычислительных машин

```
struct v4l2_fmtdesc fmtdesc = {0};
fmtdesc.type = V4L2_BUF_TYPE_VIDEO_CAPTURE;
while (ioctl(fd, VIDIOC_ENUM_FMT, &fmtdesc) == 0) {
    // fmtdesc.pixelformat — FourCC (V4L2_PIX_FMT_YUYV,
V4L2_PIX_FMT_MJPEG)
    // fmtdesc.description — текстовое описание
    fmtdesc.index++;
}
```

Разрешения и FPS для конкретного формата:

```
struct v4l2_frmsizeenum frmsize = {0};
frmsize.pixel_format = fmtdesc.pixelformat;
while (ioctl(fd, VIDIOC_ENUM_FRAMESIZES, &frmsize) == 0) {
    if (frmsize.type == V4L2_FRMSIZE_TYPE_DISCRETE) {
        // frmsize.discrete.width, frmsize.discrete.height
        struct v4l2_frmivalenum frmival = {0};
        frmival.pixel_format = fmtdesc.pixelformat;
        frmival.width = frmsize.discrete.width;
        frmival.height = frmsize.discrete.height;
        while (ioctl(fd, VIDIOC_ENUM_FRAMEINTERVALS, &frmival) == 0)
{
            if (frmival.type == V4L2_FRMIVAL_TYPE_DISCRETE) {
                // frmival.discrete.denominator /
frmival.discrete.numerator — FPS
            }
            frmival.index++;
        }
    }
    frmsize.index++;
}
```

3.6.3. Настройка формата захвата

```
struct v4l2_format fmt = {0};
fmt.type = V4L2_BUF_TYPE_VIDEO_CAPTURE;
fmt.fmt.pix.width = 640;
fmt.fmt.pix.height = 480;
fmt.fmt.pix.pixelformat = V4L2_PIX_FMT_YUYV;
fmt.fmt.pix.field = V4L2_FIELD_INTERLACED;
ioctl(fd, VIDIOC_S_FMT, &fmt);
// После установки fmt.fmt.pix.sizeimage — размер буфера кадра
```

3.6.4. Захват кадра (Memory Mapping)

```
// Запрос буферов
struct v4l2_requestbuffers req = {0};
req.count = 4;
req.type = V4L2_BUF_TYPE_VIDEO_CAPTURE;
req.memory = V4L2_MEMORY_MMAP;
ioctl(fd, VIDIOC_REQBUFS, &req);
```

Интеллектуальные устройства встраиваемых машин

```
// Отображение буферов
void* buffers[4];
for (int i = 0; i < req.count; i++) {
    struct v4l2_buffer buf = {0};
    buf.type = V4L2_BUF_TYPE_VIDEO_CAPTURE;
    buf.memory = V4L2_MEMORY_MMAP;
    buf.index = i;
    ioctl(fd, VIDIOC_QUERYBUF, &buf);
    buffers[i] = mmap(nullptr, buf.length,
                      PROT_READ | PROT_WRITE, MAP_SHARED,
                      fd, buf.m.offset);
    ioctl(fd, VIDIOC_QBUF, &buf); // Поставить в очередь
}

// Запуск захвата
enum v4l2_buf_type type = V4L2_BUF_TYPE_VIDEO_CAPTURE;
ioctl(fd, VIDIOC_STREAMON, &type);

// Получение кадра
struct v4l2_buffer buf = {0};
buf.type = V4L2_BUF_TYPE_VIDEO_CAPTURE;
buf.memory = V4L2_MEMORY_MMAP;
ioctl(fd, VIDIOC_DQBUF, &buf); // Извлечь из очереди
// buffers[buf.index] — указатель на кадр размером buf.bytesused
ioctl(fd, VIDIOC_QBUF, &buf); // Вернуть в очередь
```

3.6.5. Сохранение фотоснимка

Для формата **MJPEG** (V4L2_PIX_FMT_MJPEG) буфер уже содержит готовый JPEG-файл — достаточно записать buf.bytesused байт в файл .jpg.

Для формата **YUYV** (V4L2_PIX_FMT_YUYV) или **RGB24** (V4L2_PIX_FMT_RGB24) необходимо вручную сформировать BMP или PPM файл, либо использовать внешний кодировщик (pipe в ffmpeg допустим в данной ситуации).

3.6.6. Запись видео

Простейший способ — сохранение последовательности кадров:

- MJPEG: запись каждого кадра как отдельного JPEG в файл AVI с индексом (ручная сборка RIFF-контейнера), или использование pipe в ffmpeg для упаковки;
- RAW: запись несжатых кадров с последующей обработкой.

3.7. Linux: Обнаружение подключения / отключения камеры

Для мониторинга состояния устройств V4L2 в Linux используются:

- **inotify** на каталог /dev/ (отслеживание появления/исчезновения /dev/videoN);
- **libudev** (низкоуровневый API): мониторинг событий add / remove для подсистемы video4linux.

Интерфейсы и устройства вычислительных машин

```
#include <libudev.h>
struct udev* udev = udev_new();
struct udev_monitor* mon = udev_monitor_new_from_netlink(udev,
"udev");
udev_monitor_filter_add_match_subsystem_devtype(mon, "video4linux",
nullptr);
udev_monitor_enable_receiving(mon);
int fd = udev_monitor_get_fd(mon);
// Далее select/poll на fd
```

3.8. Linux: Скрытый режим и горячие клавиши

3.8.1. Демонизация и Unix Domain Socket

Для скрытого режима приложение может:

- запускаться как **daemon** (fork, setsid, umask), скрывая консоль;
- создавать **Unix Domain Socket** (/tmp/cam_monitor.sock) для приёма команд от GUI-клиента;
- использовать **D-Bus** (низкоуровневый sd-bus) для сигналов.

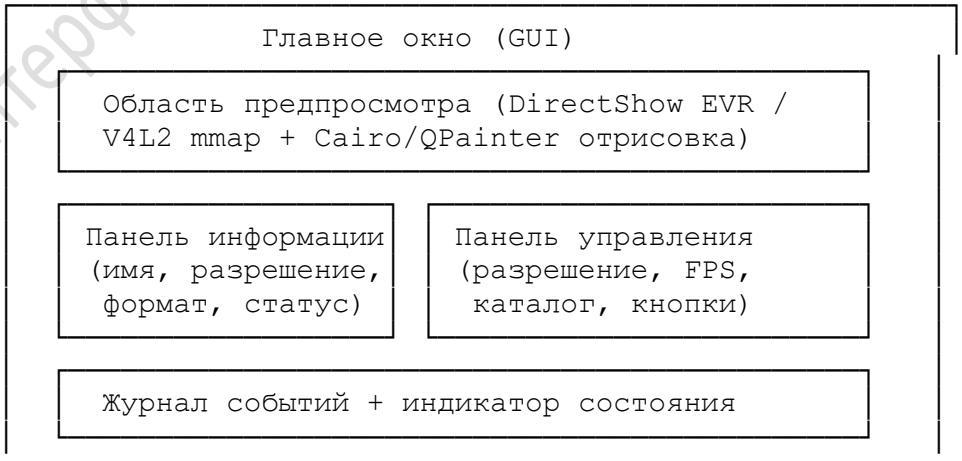
3.8.2. Глобальные горячие клавиши

В Linux среде X11 глобальные горячие клавиши можно реализовать через:

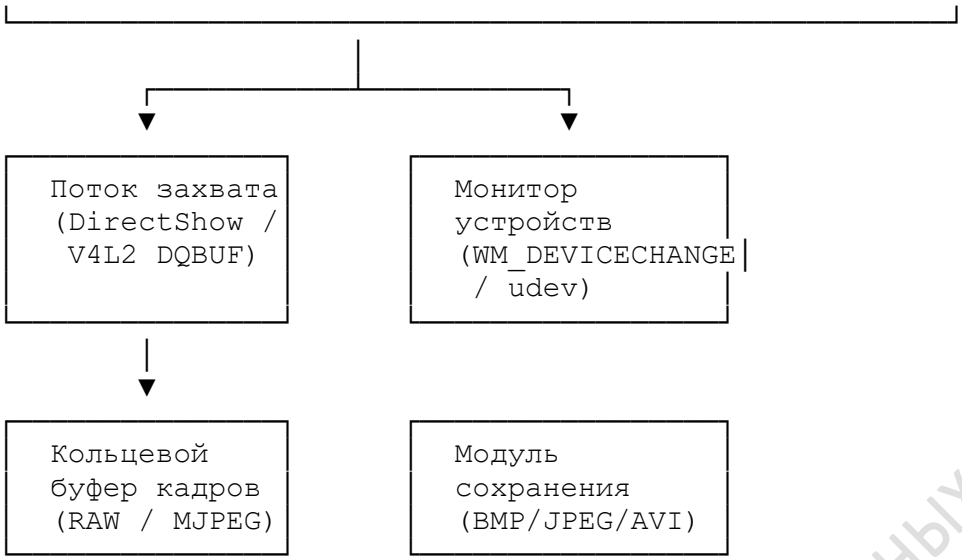
- **X11 XGrabKey** (XGrabKey(display, keycode, modifiers, root, True, GrabModeAsync, GrabModeAsync));
- **Чтение /dev/input/event*** — прямой доступ к событиям клавиатуры через evdev (требует root или членство в группе input);
- **D-Bus** — регистрация глобальных акселераторов в оконном менеджере (зависит от DE).

Рекомендуется: XGrabKey для GUI-приложений или evdev для консольных демонов.

3.9. Рекомендуемая архитектура приложения



Интерфейсы и устройства вычислительных машин



4. Варианты заданий

Вывод информации о микрофоне и аудиопотоке не производится.

ВАРИАНТЫ ДЛЯ WINDOWS

Вариант А1. Базовый видеомонитор

Реализовать приложение с одним главным окном, содержащим:

- область предпросмотра видеопотока с веб-камеры в реальном времени (через DirectShow EVR/VMR или Media Foundation);
- панель информации об устройстве: наименование, производитель, текущее разрешение, FPS, формат пикселей;
- кнопки «Подключить камеру», «Сделать снимок» (сохранение в BMP/JPEG), «Начать запись», «Остановить запись»;
- поле выбора каталога для автоматического сохранения материалов;
- индикатор состояния камеры (подключена / отключена) с цветовой индикацией;
- журнал событий с фиксацией всех операций (подключение, захват, запись, ошибки) и временными метками;
- кнопку «Сохранить отчёт» для экспорта журнала в .txt.

Дополнительно: реализовать автообновление списка камер при подключении/отключении через WM_DEVICECHANGE.

Вариант А2. Расширенная диагностика камеры

Реализовать приложение, включающее согласованные с преподавателем требования базового задания, а также:

- отдельную вкладку «Диагностика» с полным списком поддерживаемых форматов пикселей (MJPEG, YUY2, RGB24 и т.д.), разрешений и FPS для

Интерфейсы и устройства вычислительных машин

каждого формата (через IAMStreamConfig::GetStreamCaps или IMFMediaType enumeration);

- отображение технических параметров: диапазон экспозиции, баланс белого, яркость, контраст (через IAMVideoProcAmp в DirectShow);
- возможность изменения параметров видеопроцессинга (яркость, контраст, насыщенность) через ползунки с немедленным применением;
- сохранение диагностического отчёта со списком всех капабилити.

Вариант А3. Умный архив с детектором движения

Реализовать приложение с акцентом на автоматическую запись:

- базовый алгоритм детекции движения: попиксельное сравнение двух последовательных кадров (в градациях серого) с порогом различия;
- при обнаружении движения — автоматический старт записи видеофрагмента (например, 10 секунд);
- при отсутствии движения в течение заданного интервала — автоматическая остановка записи;
- отображение статуса детектора (нет движения / движение обнаружено) цветовым индикатором;
- архив записанных фрагментов с отображением даты/времени и длительности;
- настройка чувствительности детектора (порог различия пикселей, минимальная площадь объекта).

Вариант А4. Система наблюдения с несколькими камерами

Реализовать приложение, поддерживающее одновременную работу с несколькими веб-камерами:

- обнаружение всех подключённых камер (список с именами);
- отображение до 4 окон предпросмотра одновременно (сетка 2×2);
- выбор активной камеры для записи/снимка кликом по окну;
- индивидуальная настройка разрешения и FPS для каждой камеры;
- общий журнал событий с указанием источника (камеры);
- автоматическое переподключение при отключении/подключении камеры.

Вариант А5. Скрытый режим фотонаблюдения

Реализовать приложение с возможностью фонового создания снимков:

- главное окно приложения со всеми элементами базового монитора;
- кнопка «Свернуть в трей» — скрытие окна, оставление иконки в системном трее (Shell_NotifyIcon);
- регистрация глобальной горячей клавиши (RegisterHotKey, например Ctrl+Shift+F12) для создания снимка без отображения окна;

Интерфейсы устройства вычислительных машин

- при нажатии горячей клавиши в фоновом режиме — захват кадра с камеры и сохранение в выбранный каталог с именем snapshot_YYYYMMDD_HHMMSS.jpg;

- всплывающее уведомление (balloon tip) о успешном сохранении снимка;

- восстановление главного окна по двойному клику на иконке в трее;

- журналирование всех фоновых операций.

Вариант А6. Скрытый режим видеонаблюдения

Реализовать приложение с возможностью фоновой записи видео:

- главное окно с полным функционалом базового монитора;

- кнопка «Свернуть в трей» — скрытие окна;

- регистрация двух глобальных горячих клавиш: одна для старта записи, другая для остановки (например, Ctrl+Shift+R — старт, Ctrl+Shift+S — стоп);

- при старте записи в фоновом режиме — начало захвата и кодирования видеопотока без отображения окна;

- индикатор записи в трее (мигающая иконка или смена значка);

- автоматическое разбиение записи на файлы по заданному интервалу (например, каждые 5 минут);

- журналирование всех фоновых операций с временными метками.

ВАРИАНТЫ ДЛЯ LINUX

Общее для Linux-вариантов: запрещается использование высокоуровневых библиотек (OpenCV, GStreamer, FFmpeg API) без согласования с преподавателем. Рекомендуется прямое использование V4L2 ioctl, sysfs/udev для обнаружения, Cairo / Qt / GTK для отрисовки GUI.

Вариант Б1. Базовый видеомонитор (Linux)

Реализовать приложение с одним главным окном, содержащим:

- область предпросмотра видеопотока с веб-камеры в реальном времени (захват через V4L2 mmap, отрисовка через Cairo/QPainter/GTK);

- панель информации об устройстве: наименование (VIDIOC_QUERYCAP.card), текущее разрешение, FPS, формат пикселей;

- кнопки «Подключить камеру», «Сделать снимок» (сохранение MJPEG как .jpg или YUYV→PPM/BMP), «Начать запись», «Остановить запись»;

- поле выбора каталога для автоматического сохранения;

- индикатор состояния камеры (подключена / отключена);

- журнал событий с фиксацией операций и временными метками;

- кнопку «Сохранить отчёт».

Дополнительно: реализовать автообновление списка камер через udev-монитор или inotify на /dev/.

Интерфейсы и устройства вычислительных машин

Вариант Б2. Расширенная диагностика камеры (Linux)

Реализовать приложение, включающее согласованные с преподавателем требования базового задания, а также:

- отдельную вкладку «Диагностика» с полным списком поддерживаемых форматов (VIDIOC_ENUM_FMT), разрешений (VIDIOC_ENUM_FRAMESIZES) и FPS (VIDIOC_ENUM_FRAMEINTERVALS) для каждого формата;
- отображение технических параметров через VIDIOC_QUERYCTRL и VIDIOC_QUERYMENU: яркость, контраст, насыщенность, баланс белого, экспозиция, автофокус;
- возможность изменения параметров через VIDIOC_S_CTRL с ползунками;
- сохранение диагностического отчёта.

Вариант Б3. Умный архив с детектором движения (Linux)

Реализовать приложение с акцентом на автоматическую запись:

- базовый алгоритм детекции движения: попиксельное сравнение двух последовательных кадров (YUYV→Y или RGB);
- при обнаружении движения — автоматический старт записи видеофрагмента;
- при отсутствии движения — автоматическая остановка;
- цветовой индикатор статуса детектора;
- архив записанных фрагментов с датой/временем;
- настройка чувствительности.

Вариант Б4. Система наблюдения с несколькими камерами (Linux)

Реализовать приложение, поддерживающее одновременную работу с несколькими веб-камерами:

- обнаружение всех /dev/videoN через сканирование sysfs;
- отображение до 4 окон предпросмотра (сетка 2×2, отрисовка через Cairo/QPainter);
- выбор активной камеры для записи/снимка;
- индивидуальная настройка разрешения и FPS для каждой камеры (отдельные fd V4L2);
- общий журнал событий;
- автоматическое переподключение.

Вариант Б5. Скрытый режим фотонаблюдения (Linux)

Реализовать приложение с возможностью фонового создания снимков:

- главное окно с полным функционалом;

Интерфейсы и устройства вычислительных машин

- кнопка «Свернуть в трей» — скрытие окна, иконка в системном трее (AppIndicator / StatusIcon GTK или QSystemTrayIcon Qt);

- регистрация глобальной горячей клавиши (XGrabKey для X11 или чтение /dev/input/event* через evdev);

- при нажатии горячей клавиши в фоновом режиме — захват кадра и сохранение в выбранный каталог (snapshot_YYYYMMDD_HHMMSS.jpg);

- всплывающее уведомление (libnotify через D-Bus или кастомное);

- восстановление окна по двойному клику на иконке;

- журналирование фоновых операций.

Вариант Б6. Скрытый режим видеонаблюдения (Linux)

Реализовать приложение с возможностью фоновой записи видео:

- главное окно с полным функционалом;

- кнопка «Свернуть в трей»;

- регистрация двух глобальных горячих клавиш (старт/стоп записи);

- фоновый захват и сохранение видеопотока без отображения окна;

- индикатор записи в трее (мигающая иконка);

- автоматическое разбиение записи на файлы по интервалу (например, каждые 5 минут);

- журналирование всех фоновых операций.

5. Контрольные вопросы

1. Какие подсистемы видеозахвата используются в Windows (DirectShow, Media Foundation)? В чём их принципиальные отличия?

2. Как перечислить устройства видеозахвата в Windows через CLSID_SystemDeviceEnum? Опишите роль интерфейсов ICreateDevEnum, IEnumMoniker, IMoniker.

3. Как получить список поддерживаемых разрешений, FPS и форматов пикселей камеры в DirectShow (IAMStreamConfig::GetStreamCaps) и Media Foundation (IMFMediaType enumeration)?

4. Какие форматы пикселей наиболее распространены у веб-камер (RGB24, YUY2, MJPEG, NV12)? В чём их отличия с точки зрения размера кадра и сложности обработки?

5. Как организовать захват отдельного кадра в DirectShow (Sample Grabber) и Media Foundation (IMFSourceReader::ReadSample)?

6. Как сохранить захваченный кадр в файл BMP или JPEG без использования сторонних библиотек?

7. Как записать видеопоток в файл в Media Foundation (IMFSinkWriter)? Опишите основные этапы.

8. Как отслеживать подключение и отключение USB-веб-камеры в

Интерфейсы и устройства вычислительных машин

Windows? Опишите механизм WM_DEVICECHANGE и RegisterDeviceNotification.

9. Что такое скрытый режим работы приложения? Как реализовать сворачивание в системный трей и регистрацию глобальных горячих клавиш (RegisterHotKey) в Windows?

10. Какие привилегии и разрешения требуются приложению для работы с веб-камерой в Windows? Почему современные версии Windows требуют разрешения пользователя на доступ к камере?

11. Что такое Video4Linux2 (V4L2)? Какие системные вызовы используются для взаимодействия с устройством (open, ioctl, mmap, read)?

12. Как получить информацию о V4L2-устройстве (VIDIOC_QUERYCAP)? Какие поля структуры v4l2_capability содержат имя устройства, драйвер и шину?

13. Как перечислить поддерживаемые форматы пикселей (VIDIOC_ENUM_FMT), разрешения (VIDIOC_ENUM_FRAMESIZES) и частоты кадров (VIDIOC_ENUM_FRAMEINTERVALS) в V4L2?

14. Опишите механизм захвата кадра через mmap в V4L2: VIDIOC_REQBUFS, VIDIOC_QUERYBUF, mmap, VIDIOC_QBUF, VIDIOC_STREAMON, VIDIOC_DQBUF.

15. Как сохранить кадр, полученный от V4L2 в формате MJPEG? В чём особенность сохранения кадра в формате YUYV или RGB24?

16. Как организовать запись видео в Linux без использования FFmpeg/libav? Опишите подходы: последовательное сохранение JPEG-кадров, ручная сборка AVI-контейнера, pipe в ffmpeg.

17. Как отслеживать подключение и отключение веб-камеры в Linux? Сравните подходы через udev, inotify и опрос sysfs.

18. Как реализовать глобальные горячие клавиши в Linux (X11 XGrabKey, evdev /dev/input/event*)? Какие права требуются для чтения evdev?

19. Как организовать многопоточный захват видео с нескольких камер в одном приложении? Какие проблемы синхронизации при этом возникают?

20. Почему в рамках данной лабораторной работы запрещено использование OpenCV, GStreamer и FFmpeg API? Какие преимущества даёт прямое использование системных API?

6. Список рекомендуемой литературы

6.1 Основная литература (Windows)

1. Microsoft Learn. DirectShow — официальная документация по DirectShow.

2. Microsoft Learn. Media Foundation — официальная документация по Media Foundation.

Интерфейсы и устройства вычислительных машин

3. Microsoft Learn. IMFSourceReader interface — справочник по чтению медиапотоков.

4. Microsoft Learn. IMFSinkWriter interface — справочник по записи медиафайлов.

5. Microsoft Learn. WM_DEVICECHANGE message — уведомления об изменениях устройств.

6.2 Основная литература (Linux)

6. Linux Kernel Documentation. Video4Linux2 API — официальная документация V4L2.

7. Linux Kernel Documentation. V4L2 ioctl reference — справочник по ioctl-кодам V4L2.

8. man pages: v4l2(4), video4linux(7), udev(7), inotify(7), evdev(4).

9. The Linux Programming Interface. Michael Kerrisk. — No Starch Press, 2010. — Chapters 5 (Files), 14 (Memory Mappings), 63 (Alternatives and Advanced IPC).

6.3 Дополнительная литература

10. Рихтер Дж., Назаре К. **Windows via C/C++. Программирование на платформе Microsoft Windows**. — 5-е изд. — СПб.: Символ-Плюс, 2012. — Главы 7, 26 (работа с устройствами, оконные сообщения).

11. Petzold Ch. **Programming Windows**. — 5th ed. — Microsoft Press, 1998. — Chapters 5–6 (Windows messaging, GDI drawing).

12. Дунаев В. **Взаимодействие приложений с устройствами в ОС Windows**. — М.: ДМК Пресс, 2010. — Разделы 3.2–3.4 (работа с SetupAPI, устройствами мультимедиа).

13. V4L2 API Specification (PDF) — спецификация Video4Linux2 от LinuxTV.

14. FourCC Codes — справочник по кодам форматов пикселей.

15. USB Video Class (UVC) Specification — спецификация UVC для понимания капабилити веб-камер.

6.4 Справочные ресурсы

16. MSDN: RegisterHotKey — регистрация глобальных горячих клавиш.

17. MSDN: Shell_NotifyIcon — работа с системным треем.

18. libudev API Reference — документация по udev.

19. X11 XGrabKey man page — перехват клавиш в X11.

20. Linux Input Subsystem (evdev) — документация по подсистеме ввода Linux.

Интерфейсы и протоколы вычислительных машин